IPC-Signal

Concepts of Signal

A signal is a kind of (usually software) interrupt, used to communicate asynchronous events to a process. Each signal name begin with `SIG`. There is a limited list of possible signals (around 64); they are numbered, but we normally refer to them by their names. E.g.

- `SIGINT` is a signal generated when a user presses Control-C, which will terminate a program from the terminal.
- `SIGABRT` is generated when a process executes the abort function.
- `SIGSTOP` tells LINUX to pause a process to be resumed later.
- `SIGCONT` tells LINUX to resume the process paused earlier.
- `SIGSEGV` is sent to a process when it has a segmentation fault.
- `SIGKILL` is sent to a process to cause it to terminate at once.
- `SIGALRM` is generated when the timer set by the alarm function goes off.
- ....

Linux Standard Signals

The first 31 signals are standardized in LINUX:

1. SIGHUP Hangup (POSIX)
2. SIGINT Terminal interrupt (ANSI)
3. SIGQUIT Terminal quit (POSIX)
4. SIGILL Illegal instruction (ANSI)
5. SIGTRAP Trace trap (POSIX)
6. SIGIOT IOT Trap (4.2 BSD)
7. SIGBUS BUS error (4.2 BSD)
8. SIGFPE Floating point exception (ANSI)
9. SIGKILL Kill(can't be caught or ignored) (POSIX)
10. SIGUSR1 User defined signal 1 (POSIX)
11. SIGSEGV Invalid memory segment access (ANSI)
12. SIGUSR2 User defined signal 2 (POSIX)
13. SIGPIPE Write on a pipe with no reader, Broken pipe (POSIX)
14. SIGALRM Alarm clock (POSIX)
15. SIGTERM Termination (ANSI)
16. SIGSTKFLT Stack fault

Linux Standard Signal

17. SIGCHLD Child process has stopped or exited, changed (POSIX)
18. SIGCONT Continue executing, if stopped (POSIX)
19. SIGSTOP Stop executing(can't be caught or ignored) (POSIX)
20. SIGTSTP Terminal stop signal (POSIX)
21. SIGTTIN Background process trying to read, from TTY (POSIX)
22. SIGTTOU Background process trying to write, to TTY (POSIX)
23. SIGURG Urgent condition on socket (4.2 BSD)
24. SIGXCPU CPU limit exceeded (4.2 BSD)
25. SIGXFSZ File size limit exceeded (4.2 BSD)
26. SIGVTALRM Virtual alarm clock (4.2 BSD)
27. SIGPROF Profiling alarm clock (4.2 BSD)
28. SIGWINCH Window size change (4.3 BSD, Sun)
29. SIGIO I/O now possible (4.2 BSD)
30. SIGPWR Power failure restart (System V)
31. SIGSYS Bad system call

Note `kill -l` on your terminal shell to see what signals are available.

Kernel Response to the Occurrence of Signals

When the signal occurs, the process has to handle it. There are three cases:

- **Ignore the signal** – This works for most of signals, but not all. Hardware exceptions such as ”divide by 0” (with integers) cannot be ignored successfully and some signals such as SIGKILL cannot be ignored.
- **Catch and handle the exception** – The process calls a function whenever the signal occurs. The function may terminate the program gracefully or it may handle it without terminating the program.
- **Let the default action apply** – every signal has a default action. This includes:
  - ignore the signal
  - terminate the process
  - terminate and dump core
  - stop or pause the program

The signal() System Call

```
#include <signal.h>
void (*signal(int sig, void (*func)(int)))(int);
```

When `ctrl+c` is pressed, a signal SIGINT is sent to the process. The default action of this signal is to terminate the process. But this signal can also be handled.

```
// example for catching signal SIGINT

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void sig_handler(int signo){
   if (signo == SIGINT){
        sleep (1);
        printf("received SIGINT\n");
   }
}
int main(void){
    if (signal(SIGINT, sig_handler) == SIG_ERR)
         printf("\ncan't catch SIGINT\n");
     // suspends the calling process
    while(1)
        pause();
    return 0;
}
```

The signal() System Call - Example

```c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
static int myalarm = 0;
void ding (int sig) {
   myalarm = 1;
}
int main() {
   pid_t pid;
   printf("application starts \n");
   if ((pid = fork()) <0) {
      perror ("fork error");
      exit (1);
   }
   else if (pid == 0) /* child process */ {
      sleep(3);
      kill(getppid(), SIGALRM);
      exit(0);
   }
   else /* parent process */ {
      printf("waiting for myalarm \n");
      signal (SIGALRM, ding);
      pause();
      if (myalarm)
         printf("Ding\n");
      printf("done\n");
      exit (0);
   }
 }
```

https://www.gnu.org/software/libc/manual/html_node/Basic-Signal-Handling.html

The kill() and raise() System Calls

- The `kill()` system call send a signal to a specific process or a group of processes.
- The `raise()` system call allows a process to send a signal to itself.

```
#include <signal.h>
// return 0 if success; -1 on error
int kill (pid_t pid, int sig);
int raise (int sig);
```

kill() System Call Depends on The PID value

```
int kill (pid_t pid, int sig);
```

- `pid > 0`: The signal `sig` send to the process pid.
- `pid == 0`: The signal `sig` send to all process in the caller's process group.
- `pid == -1`: The signal `sig` send to all processes on the system provided that sender has permission to send.
- `pid < -1`: signal `sig` is sent to all processes in process group with `-pid`

Permission to send:

- The superuser can send a signal to any process.
- If real or effective ID of a sender's ID is the same as receiver's process, the sender send a signal to them.

The alarm() and pause() System Calls

- The `alarm()` system call is used to schedule a signal (SIGALRM) to be delivered to your process after a specified number of seconds.
- There is only one alarm clock per process.
- If a previously registered clock for the process has not yet expired when we call `alarm()`, the remaining time will be returned as a value of this function. If no alarm was set before, it returns 0.
- The SIGSTOP signal suspends the process. It cannot be handled, ignored, or blocked.
- The SIGTSTP signal is an interactive stop signal. Unlike SIGSTOP, this signal can be handled and ignored.

```
#include <unistd.h>
// Return 0 or number of seconds
unsigned int alarm (unsigned int seconds);

// It suspends the calling process until a signal is caught
// return -1 on error
int pause (void);
```

Example of alarm

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handle_alarm(int sig) {
    printf("Alarm triggered!\n");
}

int main() {
    signal(SIGALRM, handle_alarm);
    alarm(5);  // Set an alarm for 5 seconds

    printf("Waiting for alarm...\n");
    pause();  // Wait for a signal

    printf("Done.\n");
    return 0;
}
```

Example of kill() and SIGKILL

```
#include<stdlib.h>
#include<stdio.h>
#include<unistd.h>
#include<time.h>
#include<sys/wait.h>
#include<signal.h>

int main(){
  int pid = fork();
  if(pid == -1) return 1; // error
  if(pid == 0){
     while(1){
      printf("hello\n");
      sleep(1);
     }
  }else{
     sleep(1);
     kill(pid, SIGKILL); //send a signal to the process
     wait(NULL);
   }
}
```

Example for Catching User Defined and Other Signals

```
// SIGKILL, SIGSTOP cannot be caught
#include<stdio.h>
#include<signal.h>
#include<unistd.h>
void sig_handler(int signo){
    if (signo == SIGUSR1)
        printf("received SIGUSR1\n");
    else if (signo ==SIGINT)
        printf("received SIGINT:Cntl-C\n");
    else if (signo == SIGTSTP)
        printf("received SIGTSTP:Cntl-Z\n");
    else if (signo == SIGQUIT)
        printf("received SIGQUIT:Cntl-\\n");
}

int main(void){
    if (signal(SIGUSR1, sig_handler) == SIG_ERR)
        printf("\ncan't catch SIGUSR1\n");
    if (signal(SIGINT, sig_handler) == SIG_ERR)
        printf("\ncan't catch SIGINT\n");
    if (signal(SIGTSTP, sig_handler) == SIG_ERR)
        printf("\ncan't catch SIGTSTP\n");
    if (signal(SIGQUIT, sig_handler) == SIG_ERR)
        printf("\ncan't catch SIGQUIT\n");

   while(1)
        pause();
   return 0;
}
```